

Skills explained: How Skills compares to prompts, Projects, MCP, and subagents | Claude



claude.com

Since introducing Skills (<https://www.anthropic.com/news/skills>), there's been interest in understanding how the various components of Claude's agentic ecosystem work together.

Whether you're building sophisticated workflows in Claude Code (<https://www.claude.com/product/claude-code>), creating enterprise solutions with the API, or maximizing your productivity on Claude.ai (<http://claude.ai/redirect/claudedotcom.v1.f696cfd5-1f12-47f8-a535-eee101f833a1>), knowing which tool to reach for—and when—can transform how you work with Claude.

This guide breaks down each building block, explains when to use what, and shows you how to combine them for powerful agentic workflows.

Understanding your agentic building blocks

What are Skills?

<https://www.youtube.com/embed/loqpBKrNaZI>

(<https://www.youtube.com/embed/loqpBKrNaZI>).

Skills are folders containing instructions, scripts, and resources that Claude discovers and loads dynamically when relevant to a task. Think of them as specialized training manuals that give Claude expertise in specific domains—from working with Excel spreadsheets to following your organization's brand guidelines.

How Skills work: When Claude encounters a task, it scans available Skills to find relevant matches. Skills use progressive disclosure: metadata loads first (~100 tokens), providing just enough information for Claude to know when a Skill is relevant. Full instructions load when needed (<5k tokens), and bundled files or scripts load only as required.

When to use Skills: Choose Skills when you need Claude to perform specialized tasks consistently and efficiently. They're ideal for:

- **Organizational workflows:** Brand guidelines, compliance procedures, document templates

- **Domain expertise:** Excel formulas, PDF manipulation, data analysis
- **Personal preferences:** Note-taking systems, coding patterns, research methods

Example: Create a [brand guidelines Skill](https://github.com/anthropics/skills/tree/main/brand-guidelines) (<https://github.com/anthropics/skills/tree/main/brand-guidelines>) that includes your company's color palette, typography rules, and layout specifications. When Claude creates presentations or documents, it automatically applies these standards without you needing to explain them each time.

[Learn more](https://support.claude.com/en/articles/12512176-what-are-skills) (<https://support.claude.com/en/articles/12512176-what-are-skills>) about Skills and check out [our growing Skills library](https://github.com/anthropics/skills) (<https://github.com/anthropics/skills>).

What are prompts?

<https://www.youtube.com/embed/ysPbXH0LpIE>

(<https://www.youtube.com/embed/ysPbXH0LpIE>)

Prompts (<https://docs.claude.com/en/prompt-library/library>) are the instructions you provide to Claude in natural language during a conversation. They're ephemeral, conversational, and reactive—you provide context and direction in the moment.

When to use prompts: Use prompts for:

- One-off requests: "Summarize this article"
- Conversational refinement: "Make that tone more professional"
- Immediate context: "Analyze this data and identify trends"
- Ad-hoc instructions: "Format this as a bulleted list"

Example:

Please conduct a comprehensive security review of this code. I'm looking for:

1. Common vulnerabilities including:

- *Injection flaws (SQL, command, XSS, etc.)*
- *Authentication and authorization issues*
- *Sensitive data exposure*
- *Security misconfigurations*
- *Broken access control*
- *Cryptographic failures*
- *Input validation problems*
- *Error handling and logging issues*

2. For each issue you find, please provide:

- *Severity level (Critical/High/Medium/Low)*

- *Location in the code (line numbers or function names)*
- *Explanation of why it's a security risk and how it could be exploited*
- *Specific fix recommendation with code examples where possible*
- *Best practice guidance to prevent similar issues*

3. Code context: [Describe what the code does, the language/framework, and the environment it runs in – e.g., "This is a Node.js REST API that handles user authentication and processes payment data"]

4. Additional considerations:

- *Are there any OWASP Top 10 vulnerabilities present?*
- *Does the code follow security best practices for [specific framework/language]?*
- *Are there any dependencies with known vulnerabilities?*

Please prioritize findings by severity and potential impact.

Pro-tip: Prompts are your primary way of interacting with Claude, but they don't persist across conversations. For repeated workflows or specialized knowledge, consider capturing prompts as Skills or project instructions.

When to use a Skill instead: If you find yourself typing the same prompt repeatedly across multiple conversations, it's time to create a Skill. Transform recurring instructions like "review this code for security vulnerabilities using OWASP standards" or "format this analysis with executive summary, key findings, and recommendations" into Skills. This saves you from re-explaining procedures each time and ensures consistent execution.

Check out our [prompt library](https://docs.claude.com/en/prompt-library/library) (<https://docs.claude.com/en/prompt-library/library>), [prompting best practices](http://claude.com/blog/prompt-engineering-best-practices) (<http://claude.com/blog/prompt-engineering-best-practices>), or [our smart prompt maker](https://claude.ai/redirect/claudedotcom.v1.f696cfd5-1f12-47f8-a535-eee101f833a1/public/artifacts/3796db7e-4ef1-4cab-b70c-d045778f23ec) (<https://claude.ai/redirect/claudedotcom.v1.f696cfd5-1f12-47f8-a535-eee101f833a1/public/artifacts/3796db7e-4ef1-4cab-b70c-d045778f23ec>) to get started.

What are Projects?

<https://www.youtube.com/embed/nbG2DO6Xsek>
(<https://www.youtube.com/embed/nbG2DO6Xsek>)

Available on all paid Claude plans, **Projects** (<https://support.claude.com/en/articles/9517075-what-are-projects>) are self-contained workspaces with their own chat histories and knowledge bases. Each project includes a 200K context window where you can upload documents, provide context, and set custom instructions that apply to all conversations within that project.

How Projects work: Everything you upload to a project's knowledge base becomes available across all chats within that project. Claude automatically uses this context

to provide more informed, relevant responses. When your project knowledge approaches context limits, Claude seamlessly enables Retrieval Augmented Generation (RAG) mode to expand capacity by up to 10x.

When to use Projects: Choose Projects when you need:

- **Persistent context:** Background knowledge that should inform every conversation
- **Workspace organization:** Separate contexts for different initiatives
- **Team collaboration:** Shared knowledge and conversation history (on Team and Enterprise plans)
- **Custom instructions:** Project-specific tone, perspective, or approach

Example: Create a "Q4 Product Launch" project containing market research, competitor analysis, and product specifications. Every chat in this project has access to this knowledge without you needing to re-upload or re-explain the context.

When to use a Skill instead: If multiple agents or conversations need the same expertise—like security review procedures or data analysis methods—create a Skill rather than building that knowledge into individual subagents. Skills are portable and reusable, while subagents are purpose-built for specific workflows. Use Skills to teach expertise that any agent can apply; use subagents when you need independent task execution with specific tool permissions and context isolation.

[Learn \(https://support.claude.com/en/articles/9517075-what-are-projects\)](https://support.claude.com/en/articles/9517075-what-are-projects) more about Projects.

What are subagents?

[Subagents \(https://docs.claude.com/en/docs/claude-code/sub-agents\)](https://docs.claude.com/en/docs/claude-code/sub-agents) are specialized AI assistants with their own context windows, custom system prompts, and specific tool permissions. Available in Claude Code and the Claude Agent SDK, subagents handle discrete tasks independently and return results to the main agent.

How subagents work: Each subagent operates with its own configuration—you define what it does, how it approaches problems, and which tools it can access. Claude automatically delegates tasks to appropriate subagents based on their descriptions, or you can explicitly request a specific subagent.

When to use subagents: Use subagents for:

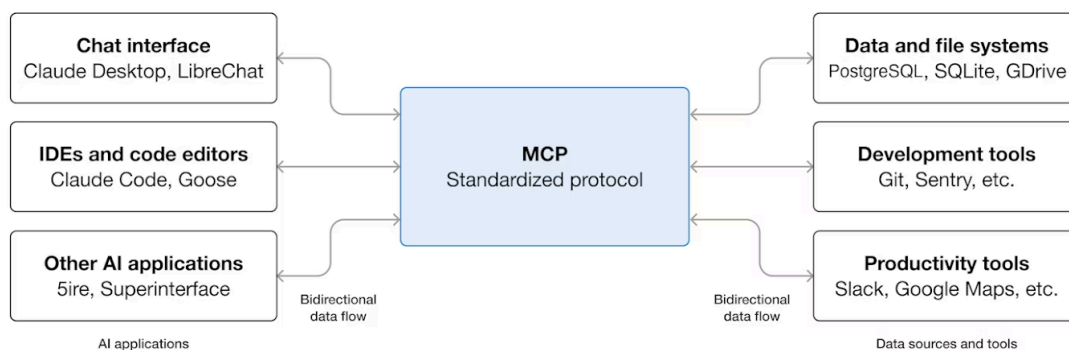
- **Task specialization:** Code review, test generation, security audits
- **Context management:** Keep the main conversation focused while offloading specialized work
- **Parallel processing:** Multiple subagents can work on different aspects simultaneously
- **Tool restriction:** Limit specific subagents to safe operations (e.g., read-only access)

Example:

When to use a Skill instead: If multiple agents or conversations need the same expertise—like security review procedures or data analysis methods—create a Skill rather than building that knowledge into individual subagents. Skills are portable and reusable, while subagents are purpose-built for specific workflows. Use Skills to teach expertise that any agent can apply; use subagents when you need independent task execution with specific tool permissions and context isolation.

[Learn more \(https://code.claude.com/docs/en/sub-agents\)](https://code.claude.com/docs/en/sub-agents) about subagents.

What is MCP?



MCP creates a universal connection layer between AI applications and your existing tools and data sources.

The Model Context Protocol (MCP) is an open standard for connecting AI assistants to external systems where data lives—content repositories, business tools, databases, and development environments.

How MCP works: MCP provides a standardized way to connect Claude to your tools and data sources. Instead of building custom integrations for each data source, you build against a single protocol. MCP servers expose data and capabilities; MCP clients (like Claude) connect to these servers.

When to use MCP: Choose MCP when you need Claude to:

- Access external data: Google Drive, Slack, GitHub, databases
- Use business tools: CRM systems, project management platforms
- Connect to development environments: Local files, IDEs, version control
- Integrate with custom systems: Your proprietary tools and data sources

Example: Connect Claude to your company's Google Drive via MCP. Now Claude can search documents, read files, and reference internal knowledge without manual uploads—the connection persists and updates automatically.

When to use a Skill instead: MCP connects Claude to data; Skills teach Claude what to do with that data. If you're explaining *how* to use a tool or follow procedures—like "when querying our database, always filter by date range first" or "format Excel reports with these specific formulas"—that's a Skill. If you need Claude to *access* the database or Excel files in the first place, that's MCP. Use both together: MCP for connectivity, Skills for procedural knowledge.

Learn more (<https://www.anthropic.com/news/model-context-protocol>) about MCP and check out documentation (<https://modelcontextprotocol.io/docs/develop/build-server>) on how to build an MCP server.

How they work together

The real power emerges when you combine these building blocks. Each serves a distinct purpose, and together they create sophisticated agentic workflows.

Comparison: choosing the right tool

Feature	Skills	Prompts	Projects	Subagents	MCP
What it provides	Procedural knowledge	Moment-to-moment instructions	Background knowledge	Task delegation	Tool connectivity
Persistence	Across conversations	Single conversation	Within project	Across sessions	Continuous connection
Contains	Instructions + code + assets	Natural language	Documents + context	Full agent logic	Tool definitions
When it loads	Dynamically, as needed	Each turn	Always in project	When invoked	Always available
Can include code	Yes	No	No	Yes	Yes
Best for	Specialized expertise	Quick requests	Centralized context	Specialized tasks	Data access

Example agentic workflow: research agent

Let's build a comprehensive research agent that combines multiple building blocks. This example shows how to assemble and activate an agent for competitive analysis.

Step 1: Set up your Project

Create a "Competitive Intelligence" project and upload:

- Industry reports and market analyses
- Competitor product documentation
- Customer feedback from your CRM
- Previous research summaries

Add project instructions:

Analyze competitors through the lens of our product strategy. Focus on differentiation opportunities and emerging market trends. Present findings with specific evidence and actionable recommendations.

Step 2: Connect data sources via MCP

Enable MCP servers for:

- Google Drive (to access shared research documents)
- GitHub (to review competitor open-source repositories)
- Web search (for real-time market information)

Step 3: Create specialized Skills

Create a "competitive-analysis" skill:

```
# My Company GDrive Navigation Skill
```

Overview

Optimized search and retrieval strategy for Meridian Tech's Google Drive structure. Use this skill to ef

Drive Organization

Top-level structure:

- `/Strategy & Planning/` - OKRs, quarterly plans, board decks
- `/Product/` - PRDs, roadmaps, technical specs
- `/Research/` - Market research, competitive intel, user studies
- `/Sales & Marketing/` - Case studies, pitch decks, campaign materials
- `/Customer Success/` - Implementation guides, success metrics
- `/Company Ops/` - Policies, org charts, team directories

Naming conventions:

- Format: `YYYY-MM-DD\_DocumentName\_vX`
- Final versions marked with `\_FINAL`
- Drafts include `\_DRAFT` or `\_WIP`

Search Best Practices

1. ****Start broad, then filter**** - Use folder context + keywords
2. ****Target document owners**** - Sales materials from Sales/, not root
3. ****Check recency**** - Prioritize documents from last 6 months for current strategy
4. ****Look for "source of truth"**** - Files with `\_FINAL`, `\_APPROVED`, or in `/Archives/Official/`

Research Agent Workflow

1. Identify topic category (product, market, customer)
2. Search relevant folder with targeted keywords
3. Retrieve 3-5 most recent/relevant documents
4. Cross-reference with `/Strategy & Planning/` for context
5. Cite sources with file names and dates

Step 4: Configure subagents (Claude Code/SDK only)

Create specialized subagents:

market-researcher subagent:


```
name: market-researcher
description: Research market trends, industry reports, and competitive landscape data. Use proactively f
tools: Read, Grep, Web-search
---
You are a market research analyst specializing in competitive intelligence.

When researching:
1. Identify authoritative sources (Gartner, Forrester, industry reports)
2. Gather quantitative data (market share, growth rates, funding)
3. Analyze qualitative insights (analyst opinions, customer reviews)
4. Synthesize trends and patterns

Present findings with citations and confidence levels.
```

technical-analyst subagent:

```
name: technical-analyst
description: Analyze technical architecture, implementation approaches, and engineering decisions. Use f
tools: Read, Bash, Grep
---
You are a technical architect analyzing competitor technology choices.

When analyzing:
1. Review public repositories and technical documentation
2. Assess architecture patterns and technology stack
3. Evaluate scalability and performance approaches
4. Identify technical strengths and limitations

Focus on actionable technical insights that inform our product decisions.
```

Step 5: Activate your research agent

Now when you ask Claude: "Analyze how our top three competitors are positioning their new AI features and identify gaps we can exploit"

Here's what happens:

1. **Project context loads:** Claude accesses your uploaded research documents and follows project instructions
2. **MCP connections activate:** Claude searches your Google Drive for recent competitor briefs and pulls GitHub data
3. **Skills engage:** The competitive-analysis Skill provides the analytical framework
4. **Subagents execute** (in Claude Code): The market-researcher gathers industry data while the technical-analyst reviews technical implementations
5. **Prompts refine:** You provide conversational guidance: "Focus especially on enterprise customers in healthcare"

The result: A comprehensive competitive analysis that draws from multiple data sources, follows your analytical framework, leverages specialized expertise, and

maintains context throughout your research project.

Common questions

How do Skills work?

Skills use progressive disclosure (<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>) to keep Claude efficient. When working on tasks, Claude first scans Skill metadata (descriptions and summaries) to identify relevant matches. If a Skill matches, Claude loads the full instructions. Finally, if the Skill includes executable code or reference files, those load only when needed.

This architecture means you can have many Skills available without overwhelming Claude's context window. Claude accesses exactly what it needs, when it needs it.

Skills vs. subagents: when to use what

Use Skills when: You want capabilities that any Claude instance can load and use. Skills are like training materials—they make Claude better at specific tasks across all conversations.

Use subagents when: You need complete, self-contained agents designed for specific purposes that handle workflows independently. Subagents are like specialized employees with their own context and tool permissions.

Use them together when: You want subagents with specialized expertise. For example, a code-review subagent can use Skills for language-specific best practices, combining the independence of a subagent with the portable expertise of Skills.

Skills vs. prompts: when to use what

Use prompts when: You're giving one-time instructions, providing immediate context, or having a conversational back-and-forth. Prompts are reactive and ephemeral.

Use Skills when: You have procedures or expertise that you'll need repeatedly. Skills are proactive—Claude knows when to apply them—and persistent across conversations.

Use them together: Prompts and Skills complement each other naturally. Use Skills to provide foundational expertise, then use prompts to provide specific context and refinement for each task.

Skills vs. Projects: when to use what

Use Projects when: You need background knowledge and context that should inform all conversations about a specific initiative. Projects provide static reference material

that's always loaded.

Use Skills when: You need procedural knowledge and executable code that activates only when relevant. Skills provide dynamic expertise that loads on-demand, saving your context window.

Use them together when: You want both persistent context and specialized capabilities. For example, a "Product Development" project containing product specs and user research, combined with Skills for creating technical documentation and analyzing user feedback data.

Key difference: Projects say "here's what you need to know." Skills say "here's how to do things." Projects provide a knowledge base you work within. Skills provide capabilities that work everywhere—any conversation, any project.

Can subagents use Skills?

Yes. In Claude Code and the Agent SDK, subagents can access and use Skills just like the main agent. This creates powerful combinations where specialized subagents leverage portable expertise.

For example, your python-developer subagent can use the pandas-analysis Skill to perform data transformations following your team's conventions, while your documentation-writer subagent uses the technical-writing skill to format API documentation consistently.

Getting started

Ready to build with Skills? Here's how to start:

Claude.ai (<https://claude.ai/>) **users:**

- Enable Skills in Settings → Features
- Create your first project at claude.ai/projects
- Try combining project knowledge with Skills for your next analysis task

API developers:

- Explore the Skills endpoint in [documentation](https://docs.anthropic.com/) (<https://docs.anthropic.com/>)
- Check out our [skills cookbook](https://github.com/anthropics/claude-cookbooks/tree/main/skills) (<https://github.com/anthropics/claude-cookbooks/tree/main/skills>)

Claude Code users:

- Install Skills via [plugin marketplaces](https://code.claude.com/docs/en/plugin-marketplaces) (<https://code.claude.com/docs/en/plugin-marketplaces>)
- Check out our [skills cookbook](https://github.com/anthropics/claude-cookbooks/tree/main/skills) (<https://github.com/anthropics/claude-cookbooks/tree/main/skills>)